

Express.js is a fast, minimalist, and highly flexible web application framework for Node.js. An Express application operates primarily as a routing and middleware pipeline, where each incoming HTTP request travels through a sequential series of middleware function calls during the request-response cycle.

At the heart of this pipeline is **middleware**, which serves as the primary building block of any Express application. Middleware functions are defined by their unique access to three parameters: the request object (req), the response object (res), and the next function in the application's stack. These functions can execute custom code, alter the request or response objects, end the request-response cycle entirely, or call `next()` to pass control down the chain. Express categorizes middleware into five distinct types: application-level, router-level, error-handling, third-party, and built-in. Among the **built-in middleware** functions, `express.json()` and `express.urlencoded()` are essential for parsing incoming request bodies into readable JavaScript objects populated directly onto `req.body`.

As applications scale, maintaining all endpoints within a single-entry file quickly becomes unmanageable. To address this, **modular routing** is achieved through **`express.Router()`**. The router acts as a self-contained "mini-application" capable of handling its own routes and specific middleware, while remaining isolated to particular segments of the parent application. It supports configurations like `caseSensitive` and `mergeParams` (which preserves parameters from parent routes) to offer robust control over nested resource hierarchies. Additionally, the router utilizes control-flow triggers like `next('route')` to bypass remaining middleware in a local route stack, or `next('router')` to escape the router instance entirely and pass control back to the parent application. This separation of concerns ensures that route files remain clean, manageable, and highly readable as the codebase grows.

In Express 5, both synchronous and asynchronous errors are caught automatically during request processing. Custom error-handling middleware is distinguished from standard middleware by its strict signature of exactly four parameters: (err, req, res, next). When an error is encountered and passed to `next(err)`, Express skips all remaining regular routes and middleware, routing the exception straight to these error-handling blocks. This enables developers to easily separate business logic from error logging, categorize errors with specialized status codes, and return clean, client-facing JSON responses rather than exposing vulnerable HTML stack traces.